A COMPREHENSIVE CASE STUDY AND FORMAL DESIGN OF PUSHDOWN AUTOMATA AND CONTEXT-FREE GRAMMARS FOR REAL-TIME COMPUTATIONAL PARSING SYSTEMS

Submitted By:

Bavaneesh M (RA2411026050009)

Formal Language Automata

In Partial Fulfilment of the Needs
for the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING WITH SPECIALIZATION IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



SCHOOL OF COMPUTING
FACULTY OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
TIRUCHIRAPPALLI, 621105
November 2026

TABLE OF CONTENTS

Chapter 1: Theoretical Foundations of Context-Free Languages & Pushdown Automata . Page 3

Chapter 2: The Inadequacy of Finite Automata for Hierarchical Languages (Formal Proofs) . Page 5

Chapter 3: Case Study 1 — Balanced Parentheses & Delimiter Syntax Verification .. Page 7

Chapter 4: Case Study 2 — Arithmetic Expression Syntax Validation & Ambiguity Resolution . Page 10

Chapter 5: Case Study 3 — Symmetrical Palindrome Recognition (L = {w c w^R}) .... Page 13

Chapter 6: Software Architecture & Python Simulation Engine Implementation ..... Page 15

Chapter 7: Comparative Analysis, Computational Complexity & Industrial Implications . Page 17

Chapter 8: Conclusion & Future Scope ..... Page 19

References & Academic Bibliography ..... Page 20

ABSTRACT

In computer science, formal language theory establishes the mathematical principles governing syntax parsing, lexical evaluation, compiler architecture, and protocol verification. While regular languages recognized by Finite Automata (FA) are optimal for bounded lexical structures, they fail to model languages requiring hierarchical nesting, unbounded matching, or symmetrical structures. This case study delivers an exhaustive formal, empirical, and architectural investigation into Type-2 Context-Free Languages (CFL), Context-Free Grammars (CFG), and Pushdown Automata (PDA).

We systematically analyze three real-world computational case studies: (1) Balanced Parentheses and Delimiter Verification in Compiler Parsing, (2) Arithmetic Expression Syntax Validation and Ambiguity Resolution, and (3) Palindrome String Recognition over Symmetrical Alphabets. For each domain, we provide formal mathematical 4-tuple and 7-tuple specifications, transition mechanics, Instantaneous Description (ID) stack evolution proofs, and comprehensive test suite evaluations (30/30 tests passed, 100% accuracy). Furthermore, we prove why Finite Automata are mathematically insufficient using the Pumping Lemma for Regular Languages and demonstrate compiler ambiguity elimination through grammar stratification.

CHAPTER 1: THEORETICAL FOUNDATIONS OF CONTEXT-FREE LANGUAGES & PUSHDOWN AUTOMATA

1.1 The Chomsky Hierarchy of Formal Grammars

In 1956, Noam Chomsky formulated a structural hierarchy classifying formal grammars into four distinct computational tiers based on the generative constraints imposed upon production rules. In this hierarchy, Context-Free Languages (Type-2) occupy the critical middle ground between computationally weak Regular Languages (Type-3) and unbounded Context-Sensitive/Recursively Enumerable systems (Type-1 and Type-0).

Type-3 Regular Grammars restrict production rules to strictly linear forms (A -> aB or A -> a). While finite state machines efficiently decide regular languages in O(n) time and O(1) space, their lack of dynamic auxiliary memory prevents them from recognizing nested, recursive, or mirrored structures.

Type-2 Context-Free Grammars liberate productions to the form A -> alpha, where A is a single non-terminal variable and alpha is any arbitrary combination of variables and terminals. This single structural property enables the expression of recursive hierarchical syntax, which forms the grammatical foundation of almost all modern programming languages, including C, C++, Java, and Python.

1.2 Formal Definition of Context-Free Grammars (CFG)

Formally, a Context-Free Grammar is defined as a mathematical 4-tuple:

$G = (V, \Sigma, R, S)$
Where:
1. V is a finite, non-empty set of non-terminal characters or syntactic variables.
2. Σ is a finite set of terminal characters disjoint from V ($V \cap \Sigma = \emptyset$).
3. R is a finite set of production rules mapping $V \to (V \cup \Sigma)^*$.
4. $S \in V$ is the designated start symbol from which all syntactic derivations originate.

A string $w \in \Sigma^*$ is said to be generated by grammar G (denoted $w \in L(G)$) if there exists a valid derivation sequence beginning at S and yielding w through repeated application of rewrite rules in R ($S \Rightarrow^* w$).

1.3 Pushdown Automata Architecture & 7-Tuple Model

A Pushdown Automaton (PDA) is the computational machine equivalent of a Context-Free Grammar. Mechanically, a PDA extends a classical finite state automaton with an auxiliary, potentially unbounded memory tape governed by a Last-In, First-Out (LIFO) stack discipline.

Mathematically, a Pushdown Automaton is defined as a 7-tuple:

$M = (Q, \Sigma, \Gamma, \delta, q0, Z0, F)$
Where:
1. Q is a finite set of internal control states.
2. Σ is the finite input alphabet.
3. Γ is the finite stack alphabet representing symbols that may be stored on the LIFO stack.
4. δ is the transition function: $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$.
5. $q0 \in Q$ is the unique start state.
6. $Z0 \in \Gamma$ is the designated start stack symbol present on the stack at machine initialization.
7. $F \subseteq Q$ is the set of accepting or final states.

1.4 Acceptance Criteria & Equivalence

A Pushdown Automaton can accept input strings through two mathematically equivalent mechanisms:
1. Acceptance by Final State (L(M)): The machine consumes the entire input string and halts in a designated final state $q \in F$, irrespective of remaining stack content.
2. Acceptance by Empty Stack (N(M)): The machine consumes the entire input string and completely empties its stack (including the initial symbol Z0), irrespective of the state in which it halts.

Theorem: For every language L accepted by a PDA M1 by final state, there exists an equivalent PDA M2 that accepts L by empty stack, and vice versa: $L(M1) = N(M2) = L$. In our implementations, acceptance by final state is utilized to provide clear, deterministic halting states.

1.5 Determinism vs. Non-Determinism (DPDA vs. NPDA)

A Pushdown Automaton is Deterministic (DPDA) if for every configuration (state q, input symbol a, and top stack symbol X), at most one transition choice exists, and if an epsilon-transition is defined, no input-consuming transition is available. Unlike Finite Automata where NFAs and DFAs are equivalent in generative power (DFA ≡ NFA), Non-Deterministic Pushdown Automata (NPDA) are strictly more expressive than Deterministic Pushdown Automata (DPDA). Deterministic Context-Free Languages (DCFL) form a proper subset of Context-Free Languages (CFL): DCFL ⊂ CFL. Compilers rely strictly on DCFLs to ensure linear-time O(n) parsing without backtracking.

CHAPTER 2: THE INADEQUACY OF FINITE AUTOMATA FOR HIERARCHICAL LANGUAGES (FORMAL PROOFS)

2.1 Memory Limitations of Finite State Machines

A Finite Automaton operates with a strictly finite, immutable state space Q. Because |Q| is constant, the automaton can retain at most |Q| distinct historical conditions. Consequently, if a computational problem requires comparing two unbounded counts (e.g., verifying that the count of closing parentheses equals the count of opening parentheses, or confirming that the second half of a string reflects the reverse of the first half), a finite automaton fails whenever the input length exceeds its state count.

2.2 Mathematical Proof via the Pumping Lemma for Regular Languages

The Pumping Lemma for Regular Languages provides a formal tool to prove that a language is non-regular. It states that if L is a regular language, there exists a pumping length p >= 1 such that any string s ∈ L with |s| >= p can be partitioned into three substrings s = xyz satisfying:
1. |xy| <= p
2. |y| >= 1
3. For all i >= 0, x y^i z ∈ L.

Proof 1: Non-Regularity of Balanced Parentheses Language L = {(^n)^n | n >= 0}

Assume, for contradiction, that L_paren = {(^n)^n | n >= 0} is regular. Let p be the pumping length granted by the lemma. Select the test string s = (^p)^p. Clearly, s ∈ L and |s| = 2p >= p.

According to the lemma, s can be split as s = xyz where |xy| <= p and |y| >= 1. Because the first p characters of s consist entirely of '(', the substring xy lies wholly within the initial block of opening parentheses. Therefore, y must be composed entirely of opening parentheses: y = (^k for some integer k >= 1.

Now, apply the pumping property for i = 2: s' = x y^2 z = (^(p+k))^p. The resulting string s' contains p + k opening parentheses but only p closing parentheses. Since k >= 1, p + k ≠ p, which means s' is unbalanced and s' ∉ L. This directly contradicts the Pumping Lemma. Hence, the assumption that L_paren is regular is false. L_paren is strictly non-regular.

Proof 2: Non-Regularity of Palindromes L = {w c w^R | w ∈ {a, b}*}

Assume, for contradiction, that L_palin = {w c w^R | w ∈ {a,b}*} is regular with pumping length p. Choose string s = a^p b c b a^p ∈ L, where |s| = 2p + 3 >= p.

Since |xy| <= p, the substring y must reside strictly within the initial prefix a^p, implying y = a^k where k >= 1. Pumping with i = 0 yields s' = x y^0 z = a^(p-k) b c b a^p. The prefix before marker 'c' now contains p - k 'a's, while the suffix after 'c' contains p 'a's. Since p - k ≠ p, the string s' is no longer symmetrical and s' ∉ L. This contradiction establishes that palindrome recognition cannot be solved by any Finite Automaton.

2.3 The Pushdown Automaton Resolution

Pushdown Automata resolve this computational limitation by pairing the finite state machine with an unbounded LIFO stack. By pushing tokens onto the stack during an input ingestion phase and popping them during a matching phase, the PDA performs exact counting across arbitrary nesting depths using only O(1) state transitions.

CHAPTER 3: CASE STUDY 1 — BALANCED PARENTHESES & DELIMITER SYNTAX VERIFICATION

3.1 Practical Context & Industrial Relevance

In integrated development environments (IDEs), syntax highlighters, and compiler front-ends, matching delimiters (parentheses, curly braces, square brackets) is the foundational syntax checking pass. A compiler must reject code containing unclosed scopes before attempting intermediate representation (IR) code generation.

3.2 Target Language Definition

The language L_paren consists of all strings over the terminal alphabet $\Sigma = \{(,)\}$ such that every opening delimiter has a corresponding closing delimiter in proper sequential hierarchy.

Formal Language Definition:
L_paren = { w ∈ {(,)}* | N_((w) = N_)(w) and for every prefix u of w, N_((u) >= N_)(u) }

3.3 Context-Free Grammar Design

The production rules for balanced parentheses are recursively formulated:
S -> (S)
S -> S S
S -> ε
Here, rule S -> (S) introduces nested balancing, S -> SS provides horizontal concatenation of balanced pairs, and S -> ε terminates recursion.

3.4 PDA Formal 7-Tuple & Transition Functions

M_paren = (Q, Σ, Γ, δ, q0, Z0, F)
• Q = {q0, q_acc, q_trap}
• Σ = {(,)}
• Γ = {Z0, X}
• Start State: q0
• Start Stack Symbol: Z0
• F = {q_acc}

Complete State Transition Mechanics (δ):

Current State	Input Symbol	Top Stack Symbol	Next State & Stack Action
q0	(	Z0	(q0, XZ0) [Push X]
q0	(	X	(q0, XX) [Push X]

q0	)	X	(q0, ε) [Pop X]
q0	ε	Z0	(q_acc, Z0) [Accept]

3.5 Step-by-Step Instantaneous Description Traces

An Instantaneous Description (ID) represents the machine configuration as a triple (q, w, α), where q is the current state, w is unconsumed input, and α is the stack string from top to bottom.

Trace for Valid Nested String '(())':
1. (q0, '(())', Z0) |- (q0, '())', XZ0) [Read '(', push X]
2. (q0, '())', XZ0) |- (q0, '))', XXZ0) [Read '(', push X]
3. (q0, '))', XXZ0) |- (q0, ')', XZ0)   [Read ')', pop X]
4. (q0, ')', XZ0) |- (q0, 'ε', Z0)   [Read ')', pop X]
5. (q0, 'ε', Z0) |- (q_acc, 'ε', Z0)   [Epsilon move, ACCEPTED]

Trace for Invalid Premature Closing String '())':
1. (q0, '())', Z0) |- (q0, '))', XZ0) [Read '(', push X]
2. (q0, '))', XZ0) |- (q0, ')', Z0)   [Read ')', pop X]
3. (q0, ')', Z0) |- (q_trap, 'ε', Z0) [Attempting to pop on Z0 with ')' -> TRAP, REJECTED]

3.6 Validation Test Cases Table

Test String	Expected	PDA Output	Final State	Syntax Evaluation
()	Valid	ACCEPTED	q_acc	Single balanced pair [PASS]
(())	Valid	ACCEPTED	q_acc	Nested balanced pair [PASS]
()()	Valid	ACCEPTED	q_acc	Concatenated balanced pairs [PASS]
(((())))	Valid	ACCEPTED	q_acc	Complex deeply nested structure [PASS]
ε (empty)	Valid	ACCEPTED	q_acc	Empty string is vacuously balanced [PASS]
(	Invalid	REJECTED	q0	Unclosed opening delimiter

				[PASS]
)	Invalid	REJECTED	q_trap	Premature closing bracket [PASS]
())(	Invalid	REJECTED	q_trap	Premature close at index 2 [PASS]
((())	Invalid	REJECTED	q0	Missing one closing parenthesis [PASS]
())	Invalid	REJECTED	q_trap	Excess closing parenthesis [PASS]

CHAPTER 4: CASE STUDY 2 — ARITHMETIC EXPRESSION SYNTAX VALIDATION & AMBIGUITY RESOLUTION

4.1 Infix Expression Parsing & The Problem of Ambiguity

In compiler design, arithmetic expressions must be translated into unambiguous abstract syntax trees (AST). If a grammar permits multiple interpretations for a single mathematical statement, the compiled binary code will produce inconsistent, non-deterministic runtime results.

4.2 Demonstration of Ambiguity in the Naive Grammar

Consider the naive context-free grammar G_ambig:
E -> E + E
E -> E * E
E -> (E)
E -> id

Let us test the string: w = id + id * id.

Derivation 1 (Leftmost Derivation yielding Tree 1):
E => E * E => E + E * E => id + E * E => id + id * E => id + id * id
Interpretation: (id + id) * id. Here, addition is evaluated first, producing incorrect arithmetic semantics.

Derivation 2 (Leftmost Derivation yielding Tree 2):
E => E + E => id + E => id + E * E => id + id * E => id + id * id
Interpretation: id + (id * id). Here, multiplication is evaluated first, adhering to standard operator precedence.

Conclusion: Because the string w = id + id * id possesses two distinct parse trees, the grammar G_ambig is formally ambiguous. Compilers cannot use ambiguous grammars directly.

4.3 Grammar Stratification & Unambiguous CFG Formulation

To eliminate ambiguity, we apply grammar stratification. We establish distinct non-terminal variables for each precedence tier:
• Expression (E): Generates binary addition (+) and enforces Left-Associativity.
• Term (T): Generates binary multiplication (*) and enforces higher precedence.
• Factor (F): Generates parenthesized sub-expressions and atomic identifiers (id).

Revised Unambiguous Grammar G_unambig:
E -> E + T | T
T -> T * F | F
F -> (E) | id

In this revised grammar, multiplication (*) is located deeper in the derivation hierarchy, ensuring that terms are grouped and evaluated before additions. Furthermore, the left-recursive production E -> E + T enforces left-to-right evaluation.

4.4 Parsing Implementation & Test Verification

Expression	Expected	Parser Status	Semantic / Precedence Evaluation
id	Valid	ACCEPTED	Atomic operand correctly identified [PASS]
id+id	Valid	ACCEPTED	Binary addition syntax valid [PASS]
id*id+id	Valid	ACCEPTED	Multiplication evaluated before addition [PASS]
(id+id)*id	Valid	ACCEPTED	Parentheses override operator precedence [PASS]
id*(id+id*id)	Valid	ACCEPTED	Multi-tier nested expression valid [PASS]
id+	Invalid	REJECTED	Binary operator missing right operand [PASS]
+id	Invalid	REJECTED	Binary operator missing left operand [PASS]
id*+id	Invalid	REJECTED	Consecutive binary operators without operand [PASS]
(id+id	Invalid	REJECTED	Unclosed opening parenthesis [PASS]
id id	Invalid	REJECTED	Juxtaposed operands missing binary operator [PASS]

CHAPTER 5: CASE STUDY 3 — SYMMETRICAL PALINDROME RECOGNITION (L = {w c w^R})

5.1 Theoretical & Engineering Context

Palindrome recognition models symmetrical data verification, checksum validation, and biological DNA sequence reflection. While even-length palindromes without a center marker require non-deterministic guessing, marked palindromes L = {w c w^R} represent a strictly Deterministic Context-Free Language (DCFL), parseable in linear time.

5.2 Formal Model Specifications

Target Language:
L_palin = { w c w^R | w ∈ {a, b}*, c is the pivot marker, w^R is the string reversal of w }

Context-Free Grammar:
S -> a S a
S -> b S b
S -> c

PDA 7-Tuple Model:
M_palin = (Q, Σ, Γ, δ, q0, Z0, F)
• Q = {q0, q1, q_acc, q_trap}
• Σ = {a, b, c}
• Γ = {Z0, A, B} (where A represents 'a' and B represents 'b')
• Start State: q0, Initial Stack Symbol: Z0, F = {q_acc}

Transition Function Specification (δ):
1. Ingestion Phase (State q0): Push incoming symbols onto the stack:
   δ(q0, a, Z0/A/B) = (q0, A Z0/A/B)
   δ(q0, b, Z0/A/B) = (q0, B Z0/A/B)
2. Pivot Phase (Marker 'c'): Switch machine state to q1 without altering stack:
   δ(q0, c, any) = (q1, any)
3. Matching Phase (State q1): Pop matching symbols from stack top:
   δ(q1, a, A) = (q1, ε)
   δ(q1, b, B) = (q1, ε)
4. Halting Phase: Accept when input is exhausted and stack holds Z0:
   δ(q1, ε, Z0) = (q_acc, Z0)

5.3 Instantaneous Description Trace for 'abbcbba'

Step-by-step configuration transitions:
1. (q0, 'abbcbba', Z0) |- (q0, 'bbcbba', AZ0)
2. (q0, 'bbcbba', AZ0) |- (q0, 'bcbba', BAZ0)

3. (q0, 'bcbba', BAZ0) |- (q0, 'cbba', BBAZ0)
4. (q0, 'cbba', BBAZ0) |- (q1, 'bba', BBAZ0) [Pivot marker 'c' processed, state shifts to q1]
5. (q1, 'bba', BBAZ0) |- (q1, 'ba', BAZ0) [Match 'b', pop B]
6. (q1, 'ba', BAZ0) |- (q1, 'a', AZ0) [Match 'b', pop B]
7. (q1, 'a', AZ0) |- (q1, 'ε', Z0) [Match 'a', pop A]
8. (q1, 'ε', Z0) |- (q_acc, 'ε', Z0) [Epsilon move, ACCEPTED]

5.4 Validation Test Cases Table

Test String	Expected	PDA Output	Final State	Symmetry Evaluation
c	Valid	ACCEPTED	q_acc	Minimal base palindrome (w = ε) [PASS]
aca	Valid	ACCEPTED	q_acc	Single character reflection (w = a) [PASS]
bcb	Valid	ACCEPTED	q_acc	Single character reflection (w = b) [PASS]
abbcbba	Valid	ACCEPTED	q_acc	Multi-character reflection (w = abb) [PASS]
baacaab	Valid	ACCEPTED	q_acc	Multi-character reflection (w = baa) [PASS]
acb	Invalid	REJECTED	q_trap	End characters do not match ('a' vs 'b') [PASS]
abcca	Invalid	REJECTED	q_trap	Reflection sequence inverted [PASS]
abbcbb	Invalid	REJECTED	q1	Length mismatch (right missing 'a') [PASS]
abc	Invalid	REJECTED	q1	Right reflection wholly missing

				[PASS]
aabaa	Invalid	REJECTED	q_trap	Pivot marker 'c' missing [PASS]

CHAPTER 6: SOFTWARE ARCHITECTURE & PYTHON SIMULATION ENGINE IMPLEMENTATION

6.1 Object-Oriented Pushdown Automaton Simulator

To simulate Pushdown Automata programmatically, we engineered a modular Python architecture in Google Colab. The core class PushdownAutomaton encapsulates states, alphabets, transition lookup dictionaries, and a dynamic LIFO list simulating stack operations.

The process(input_string) method consumes tokens linearly, tracking Instantaneous Descriptions at every step. In case of undefined transitions, the engine routes immediately to 'q_trap', guaranteeing deterministic halts.

6.2 Recursive-Descent CFG Parser Implementation

For Case Study 2, we developed the ArithmeticCFGValidator class implementing an LL(1) recursive-descent parser. The parser enforces operator precedence and associativity programmatically by mapping grammar non-terminals directly to Python recursive methods (E(), T(), F()).

6.3 Verification Suite Execution Summary

All three case studies were integrated into a unified master verification runner. Across 30 total test strings (10 per application: 5 valid, 5 invalid), the suite achieved 30 / 30 PASSED (100% verification rate).

CHAPTER 7: COMPARATIVE ANALYSIS, COMPUTATIONAL COMPLEXITY & INDUSTRIAL IMPLICATIONS

7.1 Computational Model Comparison Matrix

Attribute	Finite Automata (FA)	Pushdown Automata (PDA)	Linear Bounded (LBA)	Turing Machine (TM)
Chomsky Tier	Type-3 (Regular)	Type-2 (Context-Free)	Type-1 (Context-Sensitive)	Type-0 (Unrestricted)
Memory Structure	None (Fixed States)	Single LIFO Stack	Bounded Tape O(n)	Infinite Read/Write Tape
Counting Power	Bounded B (Constant)	Single Unbounded Count	Multiple Correlated Counts	Arbitrary Computation
Deterministic Equivalence	DFA ≡ NFA (True)	DPDA ⊂ NPDA (False)	DLBA ≡ NLBA (Open/True)	DTM ≡ NTM (True)

7.2 Time & Space Complexity

Deterministic Pushdown Automata (DPDA) process input strings in O(n) linear time, where n is input string length. Since stack operations (push and pop) execute in O(1) constant time, DPDAs achieve optimal parsing throughput. Maximum space complexity is bounded by O(n) in the worst-case nesting scenario.

7.3 Industrial Compiler & Software Engineering Applications

The principles demonstrated in these case studies are directly deployed in modern industrial software systems:

1. Compiler Generators (Yacc, Bison, ANTLR): Utilize LALR(1) and LL(k) grammars derived from context-free models to synthesize syntax parsers.
2. Data Serialization Parsers: JSON, XML, and YAML parsers use stack-based delimiter matching to validate nested hierarchies.
3. Security & Static Code Analyzers: Static analysis tools utilize stack tracking to detect memory leaks, unclosed resource handles, and bracket mismatches.

CHAPTER 8: CONCLUSION & FUTURE SCOPE

8.1 Summary of Findings

This case study has demonstrated that Pushdown Automata and Context-Free Grammars provide the rigorous mathematical framework necessary for modeling hierarchical and nested computational structures. Through balanced parentheses verification, unambiguous arithmetic expression parsing, and palindrome recognition, we have verified that LIFO stack memory extends finite automata into powerful syntax deciders capable of linear-time parsing.

8.2 Future Scope

Future extensions of this work include:
1. Automated Ambiguity Detection: Investigating heuristic approaches to detect grammar ambiguity algorithmically (noting that general CFG ambiguity is undecidable).
2. Parser Generation Pipeline: Translating the developed unambiguous grammars into production LR(1) parse tables.
3. Non-Deterministic Midpoint Guessing: Implementing backtracking simulators for unmarked palindromes L = {w w^R}.

REFERENCES & ACADEMIC BIBLIOGRAPHY

1. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Addison-Wesley.

2. Sipser, M. (2013). Introduction to the Theory of Computation (3rd ed.). Cengage Learning.

3. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Pearson Education.

4. Chomsky, N. (1956). Three models for the description of language. IRE Transactions on Information Theory, 2(3), 113-124.

5. Knuth, D. E. (1965). On the translation of languages from left to right. Information and Control, 8(6), 607-639.